Machine Learning System Design interview
3D visualization

Introduction and Overview

Group Recommendations

Google Drive Files Sharing System

Facebook Newsfeed

Ad Click Prediction System

Leaderboard System

Leaderboard System

Ad CTR Prediction Personalized Recommendations

Leaderboard System

Personalized News Feed

People You May Know

11
People You May Know

Introduction

People you may know

Figure 1: The PYMK feature

In this chapter, we will design a PYMK feature similar to LinkedIn's. The system takes a user as input and recommends a list of people to connect with.

Clarifying Requirements

Here's a typical interaction between a candidate and an interviewer.

Candidate: Is the purpose of the system to suggest friends? Which feature is it to help users discover potential connections and grow their network?

Interviewer: Yes, that's a good assumption.

Candidate: To recommend relevant connections, a huge list of factors must be considered, such as location, educational background, work experiences, existing connections, previous activities, etc. Should I focus on the most important factors, such as mutual connections and previous interactions, and leave the rest to model complexity?

Interviewer: That sounds good.

Candidate: On LinkedIn, two people are friends if one sends a "I want to be a friend of the other, or vice versa".

Interviewer: Yes, friendship is symmetrical. When someone sends a connection request to another user, the recipient needs to accept it to establish a connection. Unlike Facebook, there's no "follow" feature.

Candidate: What's the total number of users on the platform? How many of them are daily active users?

Interviewer: Let's say there exists 1 billion users and 300 million daily active users.

Candidate: How many connections does an average user have?

Interviewer: 1000 connections.

Candidate: The social graph of most users is not very dynamic, meaning their connections don't change significantly over a short period of time. Should the recommendations be personalized?

Interviewer: Yes, based on candidate's profile. This is a personalized recommendation system.

Let's summarize the problem statement. We are tasked with building a PYMK feature similar to LinkedIn's. The system takes a user as input and recommends a ranked list of potential connections as output. The primary goal for building the system is to enable users to discover new connections more easily and grow their network. There are 1 billion total users on the platform, and a user has 1000 connections on average.

Frame the problem as an ML task

Defining the ML objective

In common ML objectives in PYMK systems is to maximize the number of formed connections between users. This helps users to grow their networks quickly.

Specifying the system's input and output

The input to the PYMK system is a user, and the outputs are a list of connections ranked by relevance to the user. This is shown in Figure 2.

Figure 2: Input and output of the PYMK system

Choosing the right ML category

Let's examine two approaches commonly used to build PYMK: personalized content-based filtering (CB) and edge prediction.

Personalized CB

In this approach, we frame PYMK as a ranking problem and use a pointwise LTR to rank users. In pointwise LTR, as Figure 3.3 shows, the model takes a user-connection feature vector and learns two scores and assess the compatibility of the given pair forming a connection.

Figure 11.16: FoF of a user

According to a Meta study [2], 92% of new friendships are formed via FoF. This technique uses a user's FoF to narrow down the search space.

As previously mentioned, a user has 1,000 friends on average. That means a user has 1 million (1000 × 1000) FoF, on average. This reduces the search space from 1 billion to 1 million.

Pre-compute PYMK

Let's take a step back and consider adopting online or batch predictions.

Online prediction In PYMK, online prediction refers to generating potential connections in real-time when a user loads the homepage. In this approach, we don't generate recommendations for inactive users. Since recommendations are calculated "on the fly", if computing the recommendations takes a long time, it creates a poor user experience.

Figure 11.17: Online prediction in PYMK

Batch prediction Batch prediction means the system pre-computes potential connections for all users and stores them in a database. When a user loads the homepage, we fetch pre-computed recommendations directly, so from the end user's standpoint, the recommendation is instantaneous. The downside of batch prediction is that we may end up with unnecessary computations. Imagine 20% of users log in daily. If we generate recommendations for every user daily, then the computing power used to generate 80% of recommendations will be wasted.

Figure 11.18: Batch prediction in PYMK

Which option do we choose: online or batch? We recommend batch prediction for two reasons. First, based on the requirements gathered, there are 300 million daily active users. Computing PYMK for all 300 million users on the fly may be too slow for a quality user experience.

Second, as the social graph in PYMK does not evolve quickly, the pre-computed recommendations remain relevant for an extended period. For example, we can keep PYMK recommendations for seven days and then re-compute them. The time window can be shortened (for instance, by one day) for newer users because their networks tend to grow faster.

In a social network, a user may not want to see the same set of recommended connections repeatedly. To support this, we can pre-compute more connections than needed and only display those a user hasn't seen before.

ML system design

Figure 11.19 shows the PYMK ML system design. The design comprises two pipelines:

- PYMK generation pipeline
- Prediction pipeline

Let's inspect each.

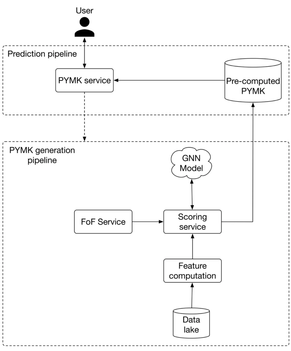

Figure 11.19: PYMK ML system design

PYMK generation pipeline

This pipeline is responsible for generating PYMK for all users and storing the results in a database. Let's take a closer look at this pipeline.

First, for a specific user, the FoF service narrows down the connections into a subset of candidate connections (2-hop neighbors). This is shown in Figure 11.20.

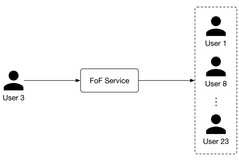

Figure 11.20: FoF service input-output

Next, the scoring service takes the candidate connections produced by the FoF service, scores each of them using the GNN model, then generates a ranked list of PYMK for the user. The PYMK is stored in a database. When a user request is made, we can simply pull their individual PYMK list directly from the database. This flow is shown in Figure 11.21.

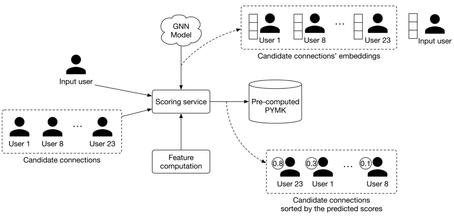

Figure 11.21: Scoring service input-output

Prediction pipeline

When a request arrives, the PYMK service first looks at the pre-computed PYMKs to see if recommendations exist. If they do, recommendations are fetched directly. If not, it sends a one-time request to the PYMK generation pipeline.

Note that what we have proposed is a simplified system. If you asked during an interview to optimize it, here are a few potential talking points:

- Pre-computing PYMK only for active users.
- Using a lightweight ranker to reduce the number of generated candidates into a smaller set before the scoring service assigns them a score.
- Using a re-ranking service to add diversity to the final PYMK list.

Other Talking Points

If there's time left at the end of the interview, here are some additional talking points:

- Personalized random walk [8] is another method often used to make recommendations. Since it's efficient, it is a helpful way to establish a baseline.
- Bias issue. Frequent users tend to have greater representation in the training data than occasional users. The model can become biased towards some groups and against others due to uneven representation in the training data. For example, in the PYMK list, frequent users might be recommended to other users at a higher rate. Subsequently, these users can make even more connections, making them even more represented in the training data [9].
- When a user ignores recommended connections repeatedly, the question arises of how to take them into account in future re-ranks. Ideally, ignored recommendations should have a lower ranking [9].
- A user may not send a connection request immediately when we recommend it to them. It may take a few days or weeks. So, when should we label a recommended connection as negative? In general, how would we deal with delayed feedback in recommendation systems [10]?

References

1. Clustering in ML. https://developers.google.com/machine-learning/clustering/overview.
2. PYMK on Facebook. https://youtu.be/Xpx5RYNTQvg?t=1823.
3. Graph convolutional neural networks. http://tkipf.github.io/graph-convolutional-networks/.
4. GraphSage paper. https://cs.stanford.edu/people/jure/pubs/graphsage-nips17.pdf.
5. Graph attention networks. https://arxiv.org/pdf/1710.10903.pdf.
6. Graph isomorphism network. https://arxiv.org/pdf/1810.00826.pdf.
7. Graph neural networks. https://distill.pub/2021/gnn-intro/.
8. Personalized random walk. https://www.youtube.com/watch?v=HbzOzUaJ_9I.
9. LinkedIn's PYMK system. https://engineering.linkedin.com/blog/2021/optimizing-pymk-for-equity-in-network-creation.
10. Addressing delayed feedback. https://arxiv.org/pdf/1907.06558.pdf.

Mark as Complete

Partner With Us

Be an Affiliate

Become a Contributor

Company & Legal

Our Team

Newsletter

Privacy Policy

Terms of Service

Do Not Sell or Share My Personal Information

Support

hi@bytebytego.com

Report a Bug

Resources

YouTube

Visual Dev Guides

Find Jobs

Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 ByteByteGo Inc. All rights reserved.